

Sprint 2

Team: *Trinix*

Additions

- Implemented a user-space program called `tree`, which recursively traverses a given directory path and displays its contents in a tree-like ASCII format, mimicking the behavior of the standard UNIX `tree` command.
 - The purpose of this command is to visually inspect the hierarchical layout of files and directories inside xv6, improving usability when exploring directory structures.
 - The `tree` command prints each file or subdirectory on a separate line, prepending it with indentation and ASCII symbols (`|`, `+`, and `|`) to reflect the tree hierarchy. This is determined using depth tracking and prefix building logic.
 - The implementation is entirely done in user space, without any kernel or file system modifications, by leveraging existing system calls:
 - `open()` to access files and directories
 - `read()` to read directory entries
 - `stat()` to determine whether an entry is a file or directory
 - `printf()` for output formatting
 - The output recursively prints contents while skipping `."` and `".."` to avoid infinite loops. The tree symbols are selected based on whether an item is the last entry in a directory.
 - Added login/signup page before opening shell with default username and password being admin and password respectively, as well as a file to keep track of all usernames and passwords thus far when signing up for future login.
-

Example Output

Directory Setup:	Command:	Output:
<pre>\$ mkdir a \$ touch a/file1 \$ mkdir a/b \$ touch a/b/file2</pre>	<pre>\$ tree</pre>	<pre>└─ a └─ file1 └─ b └─ file2</pre>

This confirms the correct recursive traversal and formatting behavior.

Documentation

The following files were added/modified:

1. tree.c
2. Makefile
3. init.c

For 1: tree.c

- This is a new user-level utility program that mimics the UNIX `tree` command.
- It recursively traverses directories and prints a visual representation of their structure.
- Used system calls: `open`, `read`, `fstat`, `stat`, `printf`.
- Handles prefix construction for indentation and branch symbols.
- Ensures path length safety and avoids visiting "." and "..".

For 2: Makefile

- Added `_tree` to the `UPROGS` section to include the program in the build.
- This allows `tree` to be compiled and loaded into `xv6` as a user program.

For 3: init.c

- Added method `signup` which opens the file "users.txt", the file responsible for saving all usernames and passwords for future use, which opens the file, prompts the user to type a username and password, checks if the username is valid (not taken) and if not, adds the username and password pair to `users.txt`.

Testing

- Manual testing was performed within `xv6` using nested directories and a mix of files and folders.
- Verified correct handling of:
 - Empty directories
 - Deep nesting

- Non-directory paths
- Output formatting was checked for correctness and clarity.

The following tests were done to check correctness:

Example of signup

```

Welcome!
1. Login
2. Signup
Choice: Signup
New Username: khaled
New Password: 123
User already exists. Choose another.
New Username:

```

Logging in with newly created username and password

```

Welcome!
1. Login
2. Signup
Choice: Login
Username: khaled
Password: 123
Login successful!
init: starting sh
$

```

Trying to login with false credentials

```

Welcome!
1. Login
2. Signup
Choice: Login
Username: khaled
Password: 132
Login failed. Try again.
Choice: _
.

```

Logging in with default username and password

```

Welcome!
1. Login
2. Signup
Choice: login
Input valid choice!
Choice: Login
Username: admin
Password: password
Login successful!
init: starting sh

```

tree in the root directory
by default

```
$ tree
.
|-- README
|-- scoutTest2
|-- scoutTest1
|-- scout
|-- cat
|-- echo
|-- forktest
|-- grep
|-- init
|-- kill
|-- ln
|-- ls
|-- mkdir
|-- rm
|-- sh
|-- stressfs
|-- usertests
|-- wc
|-- zombie
|-- tree
|-- treeTest
|-- console
+-- users.txt
```

tree after running
treeTest (we got what
we wanted and
expected)

(Note: tree runs
correctly in other
directories, such as the
a directory in our
example, however we'd
have to invoke /tree
since the method is
defined inside the root
of our xv6 program and
trying to invoke tree
without the "/" inside
directory a will yield an
error)

```
$ treeTest
$ tree
.
|-- README
|-- scountTest2
|-- scountTest1
|-- scount
|-- cat
|-- echo
|-- forktest
|-- grep
|-- init
|-- kill
|-- ln
|-- ls
|-- mkdir
|-- rm
|-- sh
|-- stressfs
|-- usertests
|-- wc
|-- zombie
|-- tree
|-- treeTest
|-- console
|-- users.txt
+-- a
    |-- b
    |   |-- c
    |   |   +-- file3
    |   +-- file2
    +-- file1
```